

Demo Script

Demo Script (5 min)

Read the `>` blocks out loud. Bold lines are what you **do**.

The packet asks the demo to cover four things. Each has its own labelled section: 1. build decisions · 2. tradeoffs · 3. measured results · 4. next iteration path

Before recording

```
cd ~/Documents/Capstone_FDE_Project && source .venv/bin/activate
rm -rf data/proposals data/decisions.json audit_log.jsonl
python data/generate_packets.py
python evals/run_evals.py           # note the numbers
.venv/bin/streamlit run app/review_app.py
```

Browser at localhost:8501. A second terminal with a big font. `data/faxes/` open in Finder so you can drag a file in. Notifications off.

0:00 - 0:15 • Opening (to camera, before sharing your screen)

“Hi, I’m Vidhanshee, and I built an intake agent for prior authorization. The short version: it catches everything missing in one pass, so providers don’t get chased three separate times for three separate things. Here’s why that matters.”

0:15 - 0:40 • The problem

“Before an insurer pays for an expensive procedure, the provider needs approval first. That’s prior authorization.

Someone checks by hand whether the request is complete. Often it isn’t, so they write back. Days pass. It returns still incomplete. They write again.

Intake is the cheapest place to fix this. Catching a missing test on day one costs nothing. Finding it on day twelve, after a denial and an appeal, costs a lot.”

0:40 – 1:10 • 1. Build decisions

Screen: the README diagram.

“It finds everything missing in one pass, writes one letter asking for all of it, routes the case, and needs a human to approve.

My main decision was where to use AI. Rules do all the checking. AI only reads language.

Field checks, eligibility, duplicates, and every routing decision are plain code. The model does two things: read the clinical notes, and write the letter.

Routing affects someone’s care timeline, so a regulator has to be able to read the rule behind it. And because the AI is contained, the whole thing runs with no API key.”

1:10 – 1:55 • See it work

```
python -m src.pipeline PA-2026-0018
```

“A request for an expensive drug. The policy needs a negative TB test, because the drug can reactivate tuberculosis.

The notes cover everything else but never mention that test. A keyword search would miss it. The model catches it, and the letter asks for exactly that, nothing more. A rubric in code enforces that before any human sees it.”

```
python -m src.pipeline --fax data/faxes/FAX-0002.txt
```

“This one started as a raw fax. It gets read into fields first. Anything the reader can’t find is left empty, never guessed.”

1:55 – 2:55 • The human gate

Screen: the review console.

“Nothing leaves this system without this screen.”

Click a case with findings: > “Each finding is labelled by what produced it: a rule, or the AI.”

Edit one line of the letter: > “Edits get captured. That’s a labelled example of where the AI fell short.”

Click Approve: > “Logged with the reviewer’s name and time. Append-only.”

Open the upload panel, drag in `FAX-0001.txt`: > “Watch it run: received, fields read, checks, duplicate check, routed, queued.”

Click Refresh queue. The case appears.

Dashboard tab: > “Every model call is metered against a budget. If it ever got stuck in a loop, it falls back to rules instead of spending money.”

2:55 – 3:55 • 3. Measured results

```
pytest tests/ -q
python evals/run_evals.py
```

“Twenty-seven tests. Twenty-nine cases across fifteen failure types. The answer key is written by the same script that breaks the data, and only the scorer can read it. A hundred percent detection, twenty-nine of twenty-nine on routing.”

Slow down. This is the best 20 seconds:

“But my favourite number came from a mistake.

An early AI run scored a perfect hundred. It was wrong. The API was rate-limiting me, and my own fallback was quietly answering half the calls with rules. The eval wasn’t testing the AI at all, and nothing told me.

So I added call integrity: how many calls the model actually answered. It caught the next broken run immediately, zero out of thirty-eight.

An eval that can’t spot its own dilution isn’t an eval.”

3:55 – 4:40 • 2. Tradeoffs and 4. what’s next

“Three things I left out on purpose. No image scanning, I read fax text and a scanner slots in front. Six procedures standing in for a full catalogue. And no auto-approval for trusted providers.

What’s next is the loop this was built for. Every reviewer edit in that audit log is a labelled example of where the AI was wrong. Feed those back into the prompts, with the eval suite as the safety net.

For production the seams are there: a data agreement, single sign-on, and the real policy catalogue.”

Close (ties back to the opener):

“That’s the intake agent. Everything’s in the repo with a one-minute setup, and it runs without any API key. Thanks for watching.”

If it breaks

Problem	Do this
Command fails or rate-limits	Add <code>PA_MODE=offline</code> in front. Say “that’s the fallback I mentioned”
App won’t load	<code>.venv/bin/streamlit run app/review_app.py --server.port 8502</code>
Odd error that doesn’t match the code	You’re on Anaconda’s Python. Use <code>.venv/bin/streamlit</code>
Lost your place	Four must-says: day-one insight · rules vs AI · human gate · the integrity bug

The 6 routes (if asked)

First match wins. Only the last two are success paths.

#	Route	When
1	<code>no_auth_required</code>	Doesn’t need prior auth. Auto-close
2	<code>duplicate_review</code>	Same member + procedure in 14 days
3	<code>eligibility_review</code>	Member not in eligibility. Our team, not the provider’s
4	<code>provider_outreach</code>	Provider must supply something. Letter drafted
5	<code>expedited_clinical_review::<specialty></code>	Complete + justified urgency
6	<code>clinical_review::<specialty></code>	Complete. Standard review

Duplicates outrank outreach so we never chase a provider about a case already in flight.

If they ask

Why not LangGraph? It’s a straight line with one pause. A framework would replace twenty lines of my code with a dependency.

Why not let AI route? I couldn’t certify it. A regulator needs to read the rule.

How do you know the letters aren't made up? A code rubric checks every draft: every finding covered, nothing invented, case ID present. One retry, then a safe template.

Where's the OCR? Deliberately none. My faxes are text, so I built the harder half. A scanner slots in front unchanged.

Isn't 100% suspicious? It should be, and the README says so. It's a consistency check against labels made by construction. The honest number is the integrity metric that caught my own eval lying.